

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет Программной инженерии и компьютерной техники

Нейроинформатика
Лабораторная работа №4

Выполнил студент
Егорова Варвара Александровна
Группа Р3323
Преподаватель: Русак Алена Викторовна

Содержание	
Задание.....	3
Ход работы.....	4
Ход эксперимента.....	4
Исходный код.....	21
Вывод.....	22

Задание

Решить задачу классификации изображений на наборе данных из лабораторной работы "Классификация изображений" с использованием предобученных моделей нейронных сетей (например, VGG16, ResNet50 и др.).

1. Провести эксперименты с замороженными весами модели и настройкой только классификатора.

2. Провести тонкую настройку сети, разморозив один или несколько слоев предобученной нейронной сети.

Ход работы

В качестве датасета использовался Fashion MNIST: 60 000 изображений для обучения и 10 000 для тестирования, размер изображений 28×28 пикселей, 10 классов одежды.

Ход эксперимента

Сначала была проведена предобработка данных: изменение формы изображений для подачи в предобученную сеть, нормализация значений пикселей и кодирование меток классов. На листинге 1 представлена предобработка датасета.

Листинг 1 - Предобработка датасета.

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
y_train = utils.to_categorical(y_train, 10)
y_test = utils.to_categorical(y_test, 10)

def preprocess_for_pretrained(x, y):
    x = tf.expand_dims(x, axis=-1)
    x = tf.image.grayscale_to_rgb(x)
    x = tf.image.resize(x, (224, 224))
    return x, y

batch_size = 32

train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train))
train_ds = train_ds.map(preprocess_for_pretrained,
num_parallel_calls=tf.data.AUTOTUNE)
train_ds =
train_ds.shuffle(1000).batch(batch_size).prefetch(tf.data.AUTOTUNE)

test_ds = tf.data.Dataset.from_tensor_slices((x_test, y_test))
test_ds = test_ds.map(preprocess_for_pretrained,
num_parallel_calls=tf.data.AUTOTUNE)
test_ds = test_ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)
```

В качестве предобученной модели использовалась VGG16 (без включения верхнего слоя классификации). На листинге 2 представлена

загрузка и дообучение модели с собственным классификатором.

Листинг 2 - Загрузка и дообучение VGG16.

```
# Загружаем базовую модель без классификатора
base_model = VGG16(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)

# Замораживаем все слои предобученной сети
base_model.trainable = False

# Добавляем свой классификатор
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation="relu")(x)
x = Dropout(0.5)(x)
outputs = Dense(10, activation="softmax")(x)

model = Model(inputs=base_model.input, outputs=outputs)

model.compile(
    optimizer=Adam(learning_rate=1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(
    train_ds,
    epochs=5,
    validation_data=test_ds
)

test_loss, test_acc = model.evaluate(test_ds)
print("Test accuracy:", test_acc)
```

Далее на листинге 3 представлен результат тестирования модели, а на рисунке 1 график изменения точности в процессе дообучения.

Листинг 3 - Результат тестирования.

```
313/313 ————— 52s 166ms/step - accuracy:
0.8459 - loss: 0.4193
```

Test accuracy: 0.840499997138977

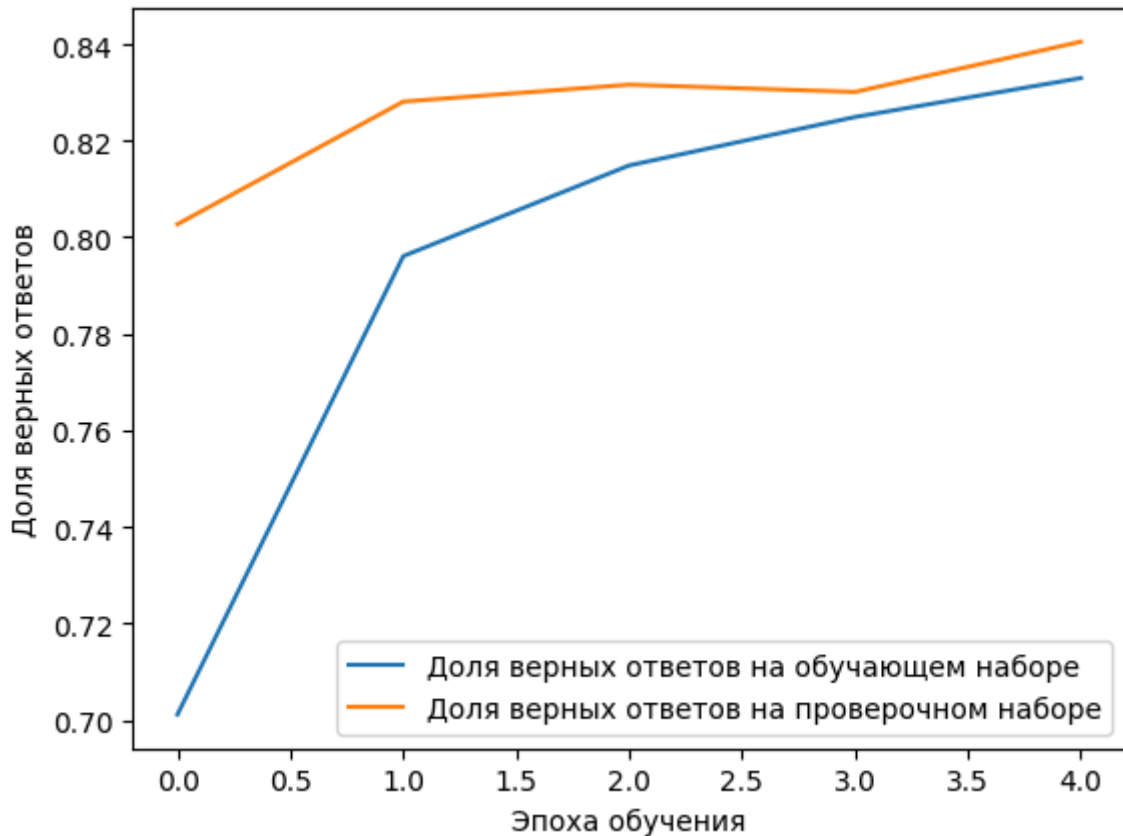


Рисунок 1 - Изменение ассигуру в процессе дообучения модели VGG16.

Далее в рамках эксперимента пробуем разморозить несколько слоёв и снизить learning rate. На листинге 4 представлена перенастройка и дообучение модели.

Листинг 4 - Перенастройка и дообучение VGG16.

```
for layer in base_model.layers[:-4]:
    layer.trainable = False

for layer in base_model.layers[-4:]:
    layer.trainable = True

from tensorflow.keras.optimizers import Adam

model.compile(
    optimizer=Adam(learning_rate=1e-5),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
```

```

)

history_finetune = model.fit(
    train_ds,
    epochs=5,
    validation_data=test_ds
)

test_loss, test_acc = model.evaluate(test_ds)
print("Test accuracy after fine-tuning:", test_acc)

```

Далее на листинге 5 представлен результат тестирования модели, а на рисунке 2 график изменения точности в процессе дообучения.

Листинг 5 - Результат тестирования.

```

313/313 _____ 52s 166ms/step - accuracy:
0.9181 - loss: 0.2248
Test accuracy after fine-tuning: 0.9158999919891357

```

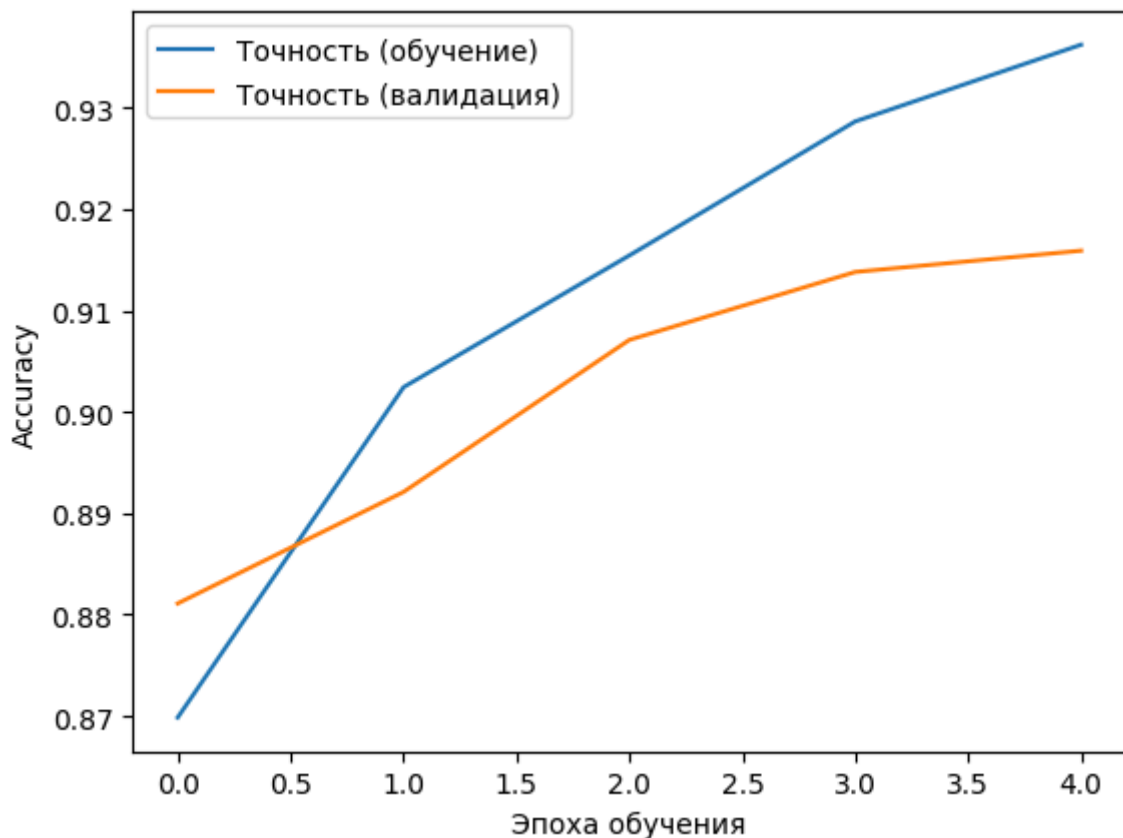


Рисунок 2 - Изменение ассигасу в процессе дообучения модели VGG16 после перенастройки.

Далее попробуем разморозить целый блок. На листинге 6 представлена перенастройка модели и ее дообучение.

Листинг 6 - Перенастройка и дообучение модели.

```
for layer in base_model.layers:
    layer.trainable = False
    if "block5" in layer.name:
        layer.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

history_block5 = model.fit(
    train_ds,
    epochs=5,
    validation_data=test_ds
)

test_loss, test_acc = model.evaluate(test_ds)
print("Test accuracy:", test_acc)
```

На листинге 7 представлен результат тестирования модели, а на рисунке 3 график изменения точности в процессе дообучения.

Листинг 7 - Результат тестирования.

```
313/313 ————— 52s 166ms/step - accuracy:
0.9272 - loss: 0.2243
Test accuracy: 0.9244999885559082
```

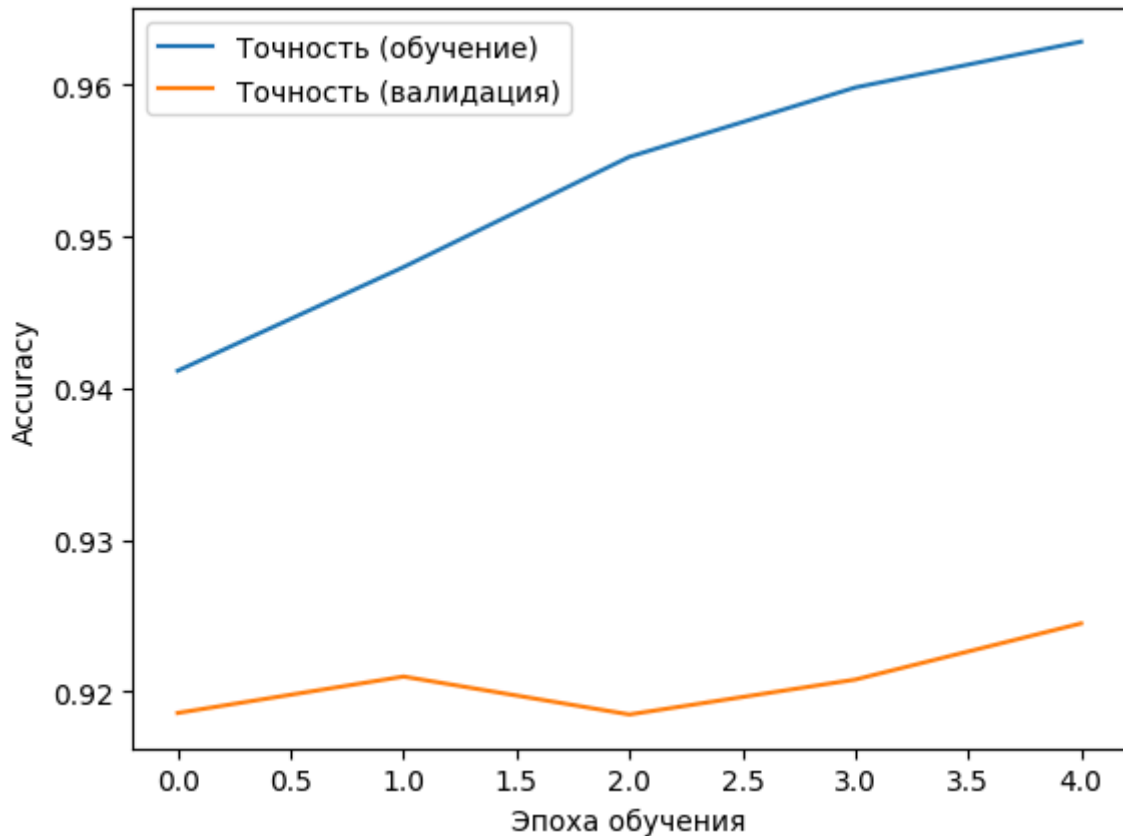



Рисунок 3 - Изменение ассигуры в процессе дообучения модели VGG16 после перенастройки.

На последнем этапе эксперимента заморозим 2 блока. На листинге 8 представлена настройка и последующее дообучение модели.

Листинг 8 - Финальная перенастройка и дообучение модели.

```
for layer in base_model.layers:
    layer.trainable = False
    if "block4" in layer.name or "block5" in layer.name:
        layer.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

history_block45 = model.fit(
    train_ds,
    epochs=5,
    validation_data=test_ds
```

```
)  
  
test_loss, test_acc = model.evaluate(test_ds)  
print("Test accuracy:", test_acc)
```

Далее на листинге 9 представлен результат тестирования модели, а на рисунке 4 график изменения точности в процессе дообучения.

Листинг 9 - Результат тестирования.

```
313/313 ————— 52s 166ms/step - accuracy:  
0.9251 - loss: 0.2627  
Test accuracy: 0.9229999780654907
```

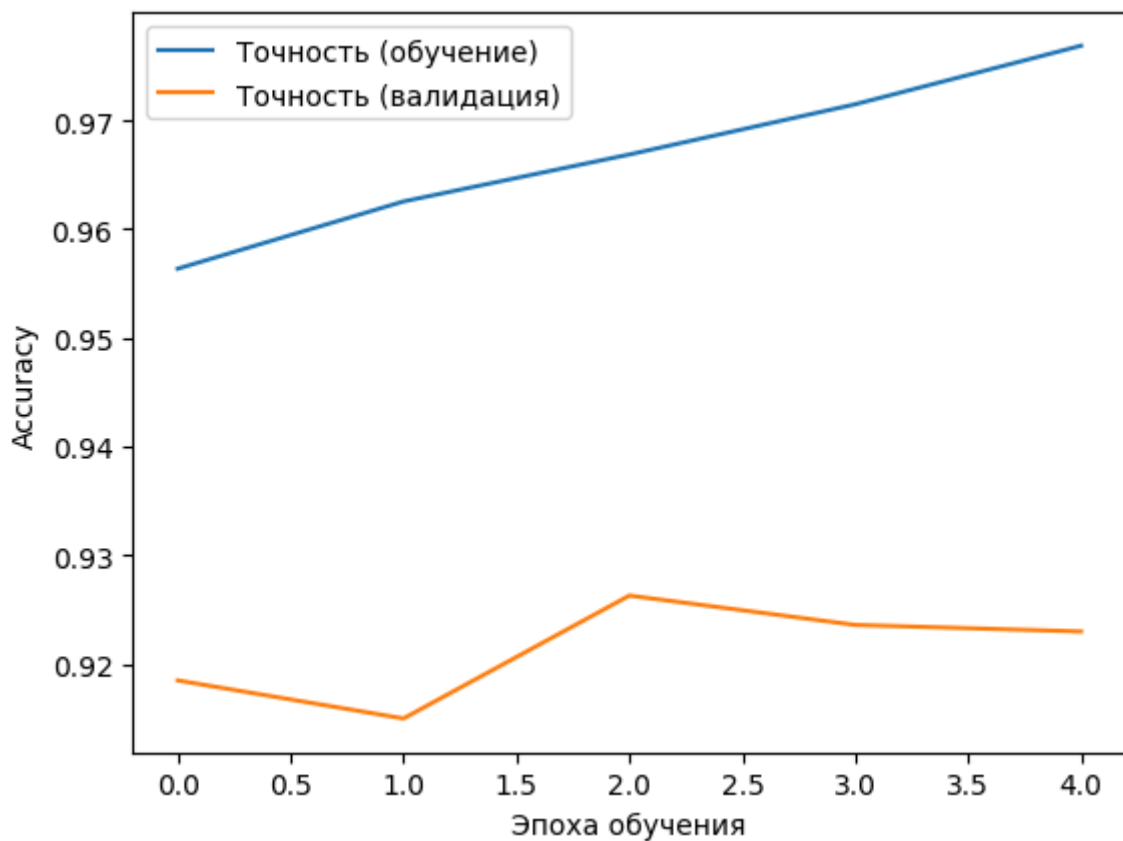


Рисунок 4 - Изменение ассигуры в процессе дообучения модели VGG16 после перенастройки.

Исходный код

Исходный полный код программы представлен в Google Colab по ссылке: [NeuroInf_lab4.ipynb](#)

Вывод

В ходе лабораторной работы была решена задача классификации изображений набора данных Fashion MNIST с использованием предобученной сверточной нейронной сети VGG16. Эксперимент с полностью замороженными весами VGG16 показал, что даже без дообучения сверточных слоев предобученная модель способна извлекать информативные признаки, обеспечивая точность классификации около 84%. Это подтверждает эффективность transfer learning при работе с относительно небольшими датасетами. Дальнейшая тонкая настройка модели за счет размораживания нескольких верхних слоев и уменьшения скорости обучения позволила значительно повысить качество классификации. Наилучшие результаты были получены при размораживании последнего блока VGG16, где точность на тестовой выборке достигла примерно 92–93%. Размораживание двух последних блоков также дало высокую точность, однако существенного прироста по сравнению с разморозкой одного блока не наблюдалось. Таким образом, результаты экспериментов показывают, что постепенное размораживание верхних слоев предобученной нейронной сети является эффективной стратегией для повышения качества модели.